Red Teaming LLM Jailbreak

Capstone Project Workflow & Task Allocation Architecture

Project Domain: AI Security & Vulnerability Assessment **Team Size:** 5 Members

Target System: LLMs (Open-weight & API-based) **Document Scope:** Workflow, Roles & Execution Roadmap

1. EXECUTIVE SUMMARY & ARCHITECTURE OVERVIEW

This document details the standardized operational workflow and task distribution for the 5-member capstone team investigating **Red Teaming and LLM Jailbreak**. The project systematically evaluates language model vulnerabilities using automated payload engines, manual prompt engineering, multimodal attack vectors, and runtime defensive guardrails.

2. TEAM ROLES & TASK DIVISION

Member 1: Red Teaming & Attack Automation Lead

Focus: Algorithmic attack generation, framework integration, automated execution

- **Framework Setup:** Configure open-source red-teaming suites (PyRIT, Garak, TAP framework).
- **Automated Jailbreaks:** Implement gradient-based and iterative algorithmic attacks (GCG, AutoDAN, Tree of Attacks with Pruning).
- **Model Wrapping:** Build standardized API connectors for running batch automated attacks against targeted LLMs (e.g., Llama-3, Mistral, GPT-4o).
- **Script Automation:** Maintain repeatable attack pipelines for regression testing against defenses.

Key Deliverables: Automated attack scripts, payload generation pipeline, integration wrappers.

Member 2: Manual Prompt Engineering & Visual Attack Specialist

Focus: Black-box prompts, obfuscation techniques, multimodal vulnerabilities

- **Black-box Tactics:** Design complex persona adoption, roleplay vectors, hypothetical framing, and multi-turn conversational bypasses.
- **Obfuscation Methods:** Evaluate encoding tricks (Base64, ROT13, cipher prompts, leetspeak) and low-resource multilingual bypasses.
- **Multimodal Attacks:** Test Visual-LLMs (VLLMs) using typographic insertion and adversarial visual noise overlay.
- **Taxonomy Mapping:** Catalog jailbreak prompts based on attack vector type and failure patterns.

Key Deliverables: Adversarial prompt library, encoding converter tools, VLLM attack suite.

Member 3: LLM Defense Architecture & Guardrails Engineer

Focus: Input/output sanitization, runtime moderation, model hardening

- **Input Guardrails:** Implement system prompt guards, intent classification layers (e.g., Llama Guard), and input perplexity filters.
- **Output Scanners:** Construct real-time response filters to detect and redact unsafe generations before returning output.
- **Defense Proxy:** Build a modular middleware proxy to intercept and defend incoming queries against Member 1 & 2's attacks.
- **System Optimization:** Conduct defensive prompt tuning and analyze trade-offs between safety and latency.

Key Deliverables: Defensive API proxy middleware, guardrail evaluation logs, sanitization ruleset.

Member 4: Vulnerability Taxonomy, Data Pipeline & Metrics Lead

Focus: Benchmark curation, automated evaluation, statistical reporting

- **Dataset Curation:** Construct a structured dataset covering OWASP Top 10 for LLMs and specific violation categories.
- **Automated Evaluation:** Develop scoring scripts utilizing LLM-as-a-Judge and semantic toxicity classifiers.
- **Metrics Tracking:** Quantify Attack Success Rate (ASR), Robustness Score under Defense, and system overhead/latency.
- **Analytics Dashboard:** Compile evaluation metrics into actionable reporting datasets for final documentation.

Key Deliverables: Standardized benchmark dataset, LLM-as-a-Judge script, evaluation analytics.

Member 5: Infrastructure, System Integration & Project Lead

Focus: Compute setup, web interface development, project coordination

- **Infrastructure Management:** Deploy local model instances (via Ollama / vLLM) and maintain API environment configs.
- **Unified Web Interface:** Build an interactive UI (Gradio / Streamlit) for live demo execution of attacks vs. defenses.
- **Integration & Testing:** Ensure seamless connection between attack modules, defense proxy, and evaluation scripts.
- **Project Oversight:** Manage Git repository, sprint milestones, capstone documentation, and final presentation slides.

Key Deliverables: Central Web Demo Dashboard, deployment architecture, final project report.

3. INTER-MEMBER COLLABORATION & OPERATIONAL MATRIX

Role	Primary Collaborators	Input Required	Output Delivered
Member 1 (Auto Attacks)	Member 4, Member 5	Benchmark Intent Prompts	Automated Execution Logs
Member 2 (Manual/ Visual)	Member 3, Member 4	Target Model Endpoint	Adversarial Prompt Repository
Member 3 (Defenses)	Member 1, Member 2	Attack Payloads	Protected API Endpoint
Member 4 (Metrics/ Eval)	All Members	Attack Outputs & Defended Outputs	ASR & Robustness Metrics
Member 5 (Infra/Lead)	All Members	Code Modules & APIs	Unified Dashboard & Documentation

4. PHASE-WISE EXECUTION TIMELINE

Phase	Focus Area	Key Milestones
Phase 1 (Weeks 1-2)	Setup & Datasets	Environment setup, local LLM deployment, benchmark dataset creation.
Phase 2 (Weeks 3-5)	Red Teaming Execution	Deploy GCG/TAP automated attacks, craft manual/visual jailbreak suite.
Phase 3 (Weeks 6-8)	Defense Implementation	Deploy input/output guardrails (Llama Guard), integrate defensive proxy.
Phase 4 (Weeks 9-10)	Evaluation & UI	Calculate ASR vs. Defended baseline, build Gradio/Streamlit demonstration.
Phase 5 (Weeks 11-12)	Documentation & Defense	Finalize capstone report, system architecture diagrams, and slide deck.

> **Note for Academic Execution:** Ensure all testing is conducted in isolated environment instances (local Ollama/vLLM or designated API sandbox environments) to satisfy safety and research ethics requirements.